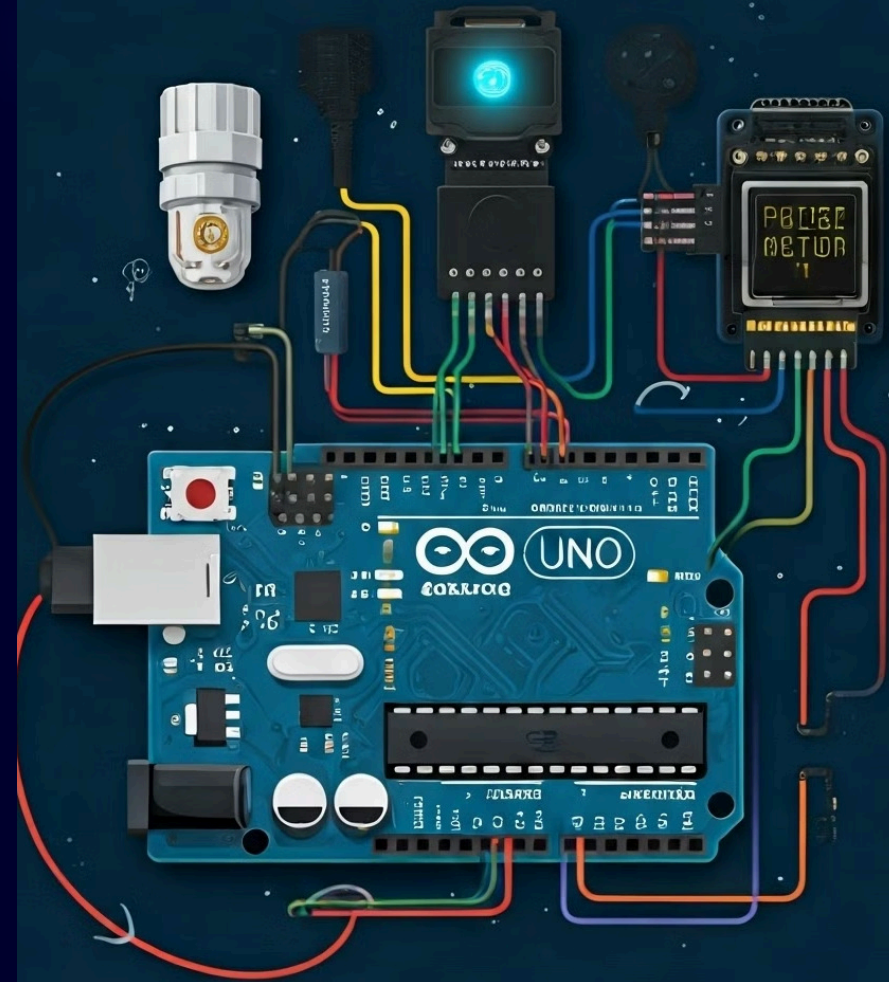


# Gas Leakage Detection System with Auto Cutoff

Innovative system for detecting and preventing dangerous gas leaks.





# Introduction to Gas Leakage Problem

Gas leaks pose serious safety hazards, potentially leading to explosions and health issues.

- 1 Accidental Leaks**  
Can occur due to faulty pipes, equipment malfunction, or natural disasters.
- 2 Health Risks**  
Exposure to gas can cause dizziness, headaches, and even death.
- 3 Property Damage**  
Explosions can cause significant damage to buildings and surrounding areas.

# Importance of Gas Leak Detection

Early detection is crucial for preventing catastrophic events and ensuring safety.

1

## Early Warning

Provides sufficient time for evacuation.

2

## Automatic Cutoff

Stops gas flow, preventing further leaks.

3

## Reduced Risk

Minimizes potential for explosions and health issues.



# Components and Sensors Used

System utilizes specific components and sensors for effective detection and control.

## Gas Sensor(MQ-2)

Detects the presence of flammable gases.

## Arduino Uno

Microcontroller for processing sensor data and controlling actuators.

## SERVO MOTOR

Automatically cuts off gas regulator when a leak is detected.

# Working Principle of the System

System works by monitoring gas levels and activating a cutoff mechanism.

1

## Gas Sensor

Detects gas concentration.

2

## Arduino Processing

Analyzes sensor data and triggers alarm.

3

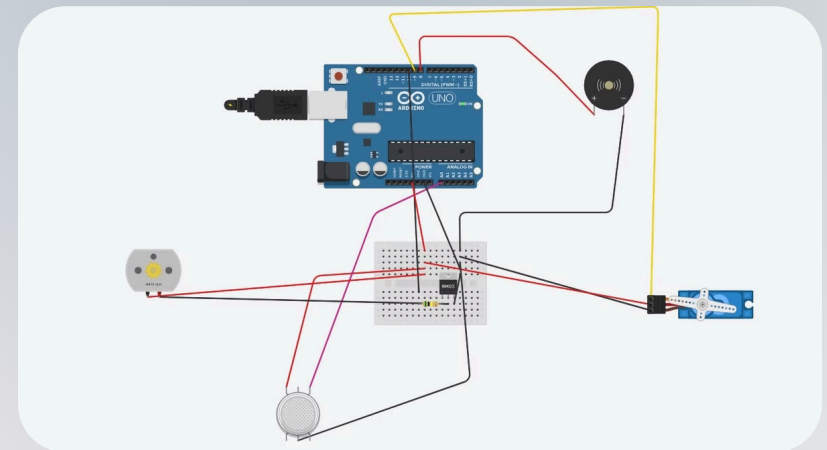
## Servo motor

Closes gas Regulator, preventing further leakage.

4

## Exhaust Fan

when regulator turns off automatically exhaust fan starts.



# Arduino Uno Programming and Integration

Arduino code manages sensor data, alarm, and turning off servo motor and start the exhaust fan

```
#include <Servo.h>
```

```
const int mq2Pin = A0; // MQ-2 sensor pin
const int buzzerPin = 8; // Buzzer pin
const int fanPin = 10; // Fan control pin (MOSFET)
const int servoPin = 9; // Servo control pin
```

```
Servo myServo; // Create a Servo object
```

```
void setup() {
  Serial.begin(9600); // Start serial communication
  pinMode(buzzerPin, OUTPUT);
  pinMode(fanPin, OUTPUT);
  myServo.attach(servoPin); // Attach the servo to the pin
  myServo.write(0); // Start with the valve closed
}
```

```
void loop() {
  int mq2Value = analogRead(mq2Pin); // Read MQ-2 sensor value
  Serial.println(mq2Value); // Print value for debugging
}
```

```
if (mq2Value > 400) { // Threshold value for LPG detection
  digitalWrite(buzzerPin, HIGH); // Activate buzzer
  digitalWrite(fanPin, HIGH); // Turn on fan
  myServo.write(180); // Open valve (cutoff)
  delay(2000); // Keep active for 10 seconds
} else {
  digitalWrite(buzzerPin, LOW); // Deactivate buzzer
  digitalWrite(fanPin, LOW); // Turn off fan
  myServo.write(0); // Close valve
}
```

```
delay(500); // Delay for stability
```

sketch\_sep09a

```
#include <Servo.h>

const int mq2Pin = A0; // MQ-2 sensor pin
const int buzzerPin = 8; // Buzzer pin
const int fanPin = 10; // Fan control pin (MOSFET)
const int servoPin = 9; // Servo control pin

Servo myServo; // Create a Servo object

void setup() {
  Serial.begin(9600); // Start serial communication
  pinMode(buzzerPin, OUTPUT);
  pinMode(fanPin, OUTPUT);
  myServo.attach(servoPin); // Attach the servo to the pin
  myServo.write(0); // Start with the valve closed
}

void loop() {
  int mq2Value = analogRead(mq2Pin); // Read MQ-2 sensor value
  Serial.println(mq2Value); // Print value for debugging

  if (mq2Value > 400) { // Threshold value for LPG detection
    digitalWrite(buzzerPin, HIGH); // Activate buzzer
    digitalWrite(fanPin, HIGH); // Turn on fan
    myServo.write(180); // Open valve (cutoff)
    delay(2000); // Keep active for 10 seconds
  } else {
    digitalWrite(buzzerPin, LOW); // Deactivate buzzer
    digitalWrite(fanPin, LOW); // Turn off fan
    myServo.write(0); // Close valve
  }

  delay(500); // Delay for stability
}
```

# Hardware Assembly and Configuration

Assembling and configuring components is a crucial step.



## Wiring Connections

Connecting sensor, Servo motor, Fan, and Arduino.



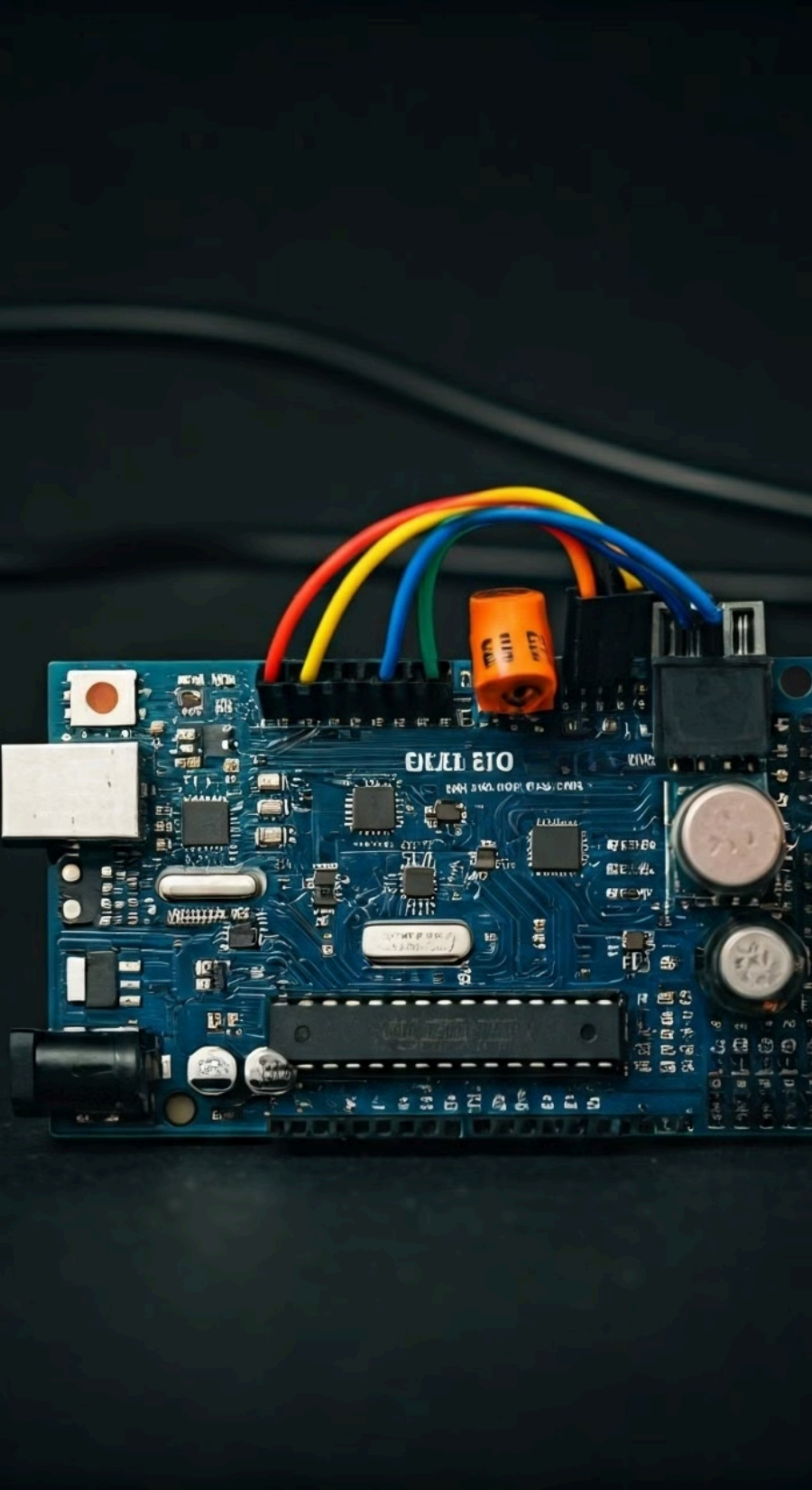
## Configuration

Setting gas threshold, alarm parameters.



## Power Supply

Connecting power source for Arduino and peripherals.



# Testing and Validation

Testing ensures system accuracy and responsiveness.

## Simulated Leaks

Using controlled gas exposure to test sensor accuracy.

## Alarm Response

Verifying alarm activation at specific gas concentrations.

## Motor Functionality

Confirming automatic gas cutoff in response to detected leaks.



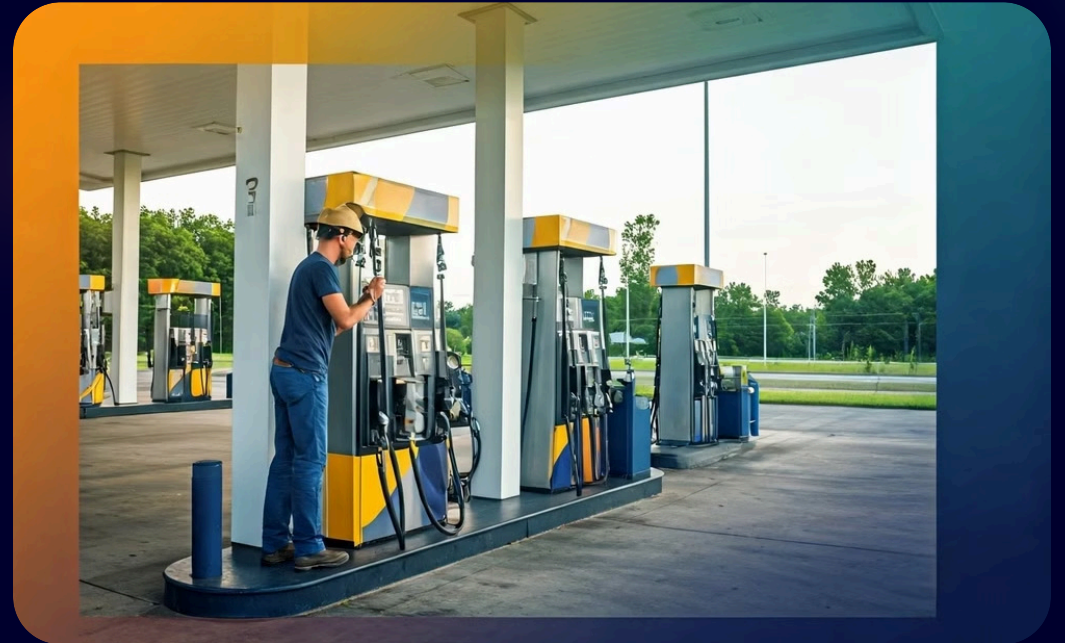
# Advantages and Applications

System offers benefits for various applications.



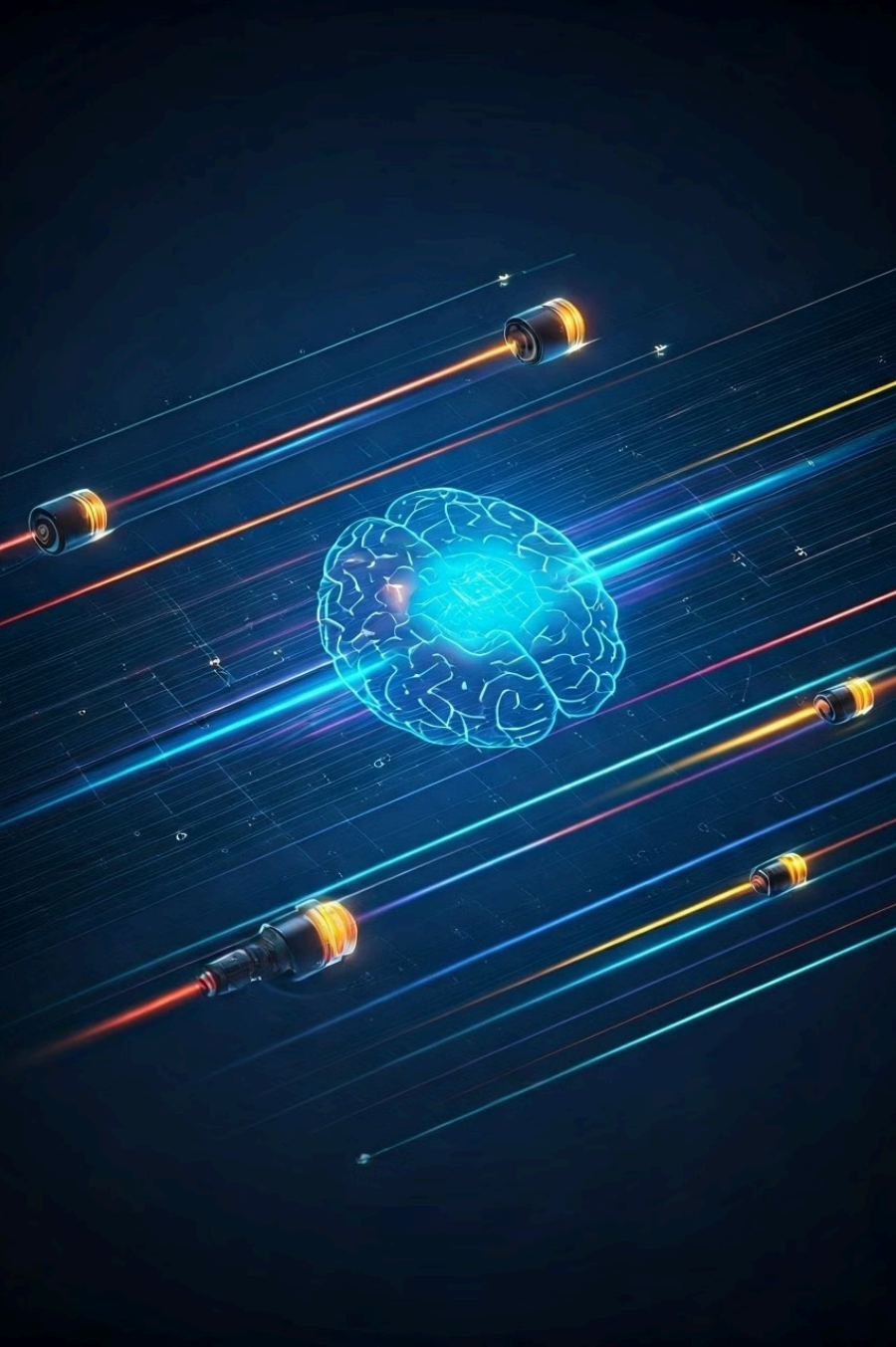
## Residential Safety

Protecting homes from gas leaks.



## Industrial Facilities

Ensuring safety in factories and refineries.



# Conclusion and Future Scope

System effectively detects and prevents gas leaks.

1

## Enhanced Security

Increased safety for homes, businesses, and industries.

2

## Future Integration

Potential for wireless communication and remote monitoring.